

Self-Constructing Agents: A Domain-Agnostic Framework for Verified Skill and Tool Persistence in Code-Acting Language-Model Agents

Ihsan Alaeddin and Sazan Khalid

Abstract

Most language-model agents operate without persistent memory: every new task begins from a blank slate, and the engineering knowledge produced during one session evaporates when the session ends. We present a domain-agnostic framework in which a code-acting agent writes, verifies, and accumulates a permanent library of executable Python skills and external tools as a by-product of normal task execution. The architecture extends the open-ended action-accumulation paradigm with five contributions: a four-gate verification pipeline that enforces syntactic, executional, static-compliance, and adversarial-validation checks; a typed distinction between in-process skills and boundary-crossing tools, each with its own verification semantics; a skill lifecycle layer that composes, evolves, and decays artifacts; an episodic memory layer; and a domain-agnostic configuration interface. We provide three forms of evidence: a controlled cold-warm experiment against the FastAPI codebase showing the warm agent achieving 2.67 out of 3.0 versus the cold agent’s 2.00, a relative improvement of 33.3 percent; an interactive entry point demonstrating live tool authoring in which the agent autonomously builds a verified `fetch_stock_price` tool against Yahoo Finance; and a worked ISO 20022 cross-border payment simulation. Notably, only one of six accumulated skills was directly invoked during the warm benchmark, indicating a multi-channel benefit deriving from episodic memory and tool-call discipline as well as direct skill reuse. We discuss limitations including small sample size, single-model evaluation, and the absence of a controlled second-domain replication.

1 Introduction

Language-model agents in production are stateless. They receive a task, reason through tool calls or code-execution steps, return an answer, and forget everything in the process. The next time a similar task appears, the entire reasoning trace must be reconstructed. The institutional knowledge

generated by an agent’s effort accumulates only in conversation logs, which no subsequent agent reads.

This is a structural limitation. In any domain where the same kinds of problems recur, a stateless agent is incapable of getting better at the work, because every session starts at the same baseline as the first. A consultant who has spent three years writing ISO 20022 message parsers becomes faster, more accurate, and more aware of edge cases over time. The agent does not. The asymmetry is not a matter of model size or prompt engineering. It is a consequence of the absence of cross-session persistent procedural memory.

This paper studies a class of agents that retain procedural memory as verified, executable Python functions and tools saved to a local persistent library. Each time the agent solves a task that produces reusable code, it writes that code as a function with metadata, runs a unit test, and persists it only if the test passes and additional checks succeed. Future tasks can retrieve, invoke, compose, and evolve these artifacts. The library is the agent’s accumulated procedural knowledge, encoded in a form that is verifiable rather than narrative.

A central architectural commitment of this work is that the framework is domain-agnostic. The orchestrator, verification pipeline, skill lifecycle, episodic memory, and MCP tool-calling system are implemented without hardcoded domain assumptions. Domain-specific configuration including compliance rules, RAG corpora, exploration tasks, and benchmark tasks is supplied through external configuration files. A user can point the framework at any codebase by providing a configuration bundle, without modifying agent code.

To ground development, we tuned an initial configuration for financial-protocol integration. SWIFT formally retired the legacy MT message format in November 2025 in favour of ISO 20022 MX. Industry surveys projected that nearly five thousand member banks would not be fully compliant in time. Eighty-three percent of payment-modernisation projects deliver late, eighty-seven percent over budget, with average direct cost of approximately \$497,000 per problematic project. A self-constructing agent that accumulates verified payment-protocol skills is, in principle, a partial solution. The skills built for one institution apply to the next. The library compounds across deployments rather than starting from zero.

The research questions of this paper address performance (does an agent with an accumulated verified skill library outperform one starting cold), mechanism (where does the benefit come

from, direct skill reuse, accumulated context, or some combination), verification (is the verification pipeline strict enough to keep the library trustworthy without suppressing skill formation), and generality (is the architecture transferable across domains).

2 Related Work

Yao et al. [1] established the standard pattern of interleaving reasoning with structured tool calls in ReAct. Reflexion [2] extended this with verbal self-reflection. Both treat the action space as a fixed library. Our framework treats it as something the agent can grow.

CodeAct [5] argues that Python code is a more expressive action representation than JSON tool calls, reducing the model calls required for multi-step tasks. The smolagents library [6] reports a 30 percent reduction. We adopt the code-as-action paradigm. Every artifact, both skills and tools, is Python code.

Voyager [3] demonstrated agent skill acquisition in Minecraft. DynaSaur [4] generalized this beyond gaming. An agent reuses an existing function or generates a new one as a Python snippet, accumulating verified-by-execution code into a growing action set. We extend this in five respects. First, a four-gate verification pipeline rather than a single execution check. Second, an explicit skill-tool typing distinction with separate verification semantics. Third, lifecycle mechanics including composition, evolution, decay, and a five-category failure-recovery taxonomy. Fourth, a domain-agnostic external configuration interface. Fifth, an episodic memory layer that injects similar past tasks into the system prompt of each new task.

Generative Agents [7] and MemGPT [8] store memory as natural-language text. Our work is a procedural-memory analogue. Knowledge is stored as provably correct, immediately invocable code rather than retrieved as descriptive text and re-interpreted by the model.

3 System Architecture

The system runs as a single Python process organised in seven coordinated components. All persistent state lives in flat files on disk that the orchestrator reads at the start of every task and writes to at the end.

The orchestration layer (`run.py`, 1,629 lines) is the central task loop. It reads tasks from a queue or interactive entry point, assembles the system prompt, and routes the LLM response to the correct downstream handler. The orchestrator owns no permanent state. It coordinates.

A single LLM, configurable by environment variable, is used at the reasoning layer. For the experiments reported here we used `gemini-2.5-flash-lite`. The model receives the assembled system prompt and produces one of four block types. `---EXECUTE---` is a Python snippet to run inline, ephemeral. `---PY SKILL---` is a Python function plus unit test, candidate for the skill library. `---MCP TOOL---` is a function plus JSON input/output schema, candidate for the tool registry. `---NEW TASK---` is a follow-up task to enqueue. A fifth signal, `---TOOL CALL---`, indicates that the agent needs live external data and pauses for the orchestrator to dispatch.

3.1 The Four-Gate Verification Pipeline

Every candidate artifact (skill or tool) passes through four sequential gates before persisting. Failure at any gate triggers a retry with the failure detail appended to the conversation thread, up to three attempts per artifact name.

Gate 1: AST validity. The artifact source is parsed via Python’s `ast.parse`. Syntax errors are common and cheap to detect. They are the most frequent cause of rejection. A common Gate 1 failure is the model truncating output mid-function due to a `max_output_tokens` ceiling set too low.

Gate 2: Executional verification. The artifact and its unit test are written to a temporary file and executed in a subprocess with a 30-second timeout. The artifact is admitted only if the subprocess exits cleanly and stdout contains the literal string `TEST PASSED`. The exact-marker requirement is deliberate. Many models write `assert`-only tests that succeed silently with no output. Requiring an explicit print disambiguates “test passed” from “test never ran.”

Gate 3: ComplianceShield. The `compliance.py` module (143 lines) performs regex-based static analysis. A block list rejects hard-coded routing-number-pattern strings, IBAN literals, hard-coded credentials, and SSN patterns. A warn list flags artifacts that reference audit-relevant identifiers (such as `account_number` or `iban`) without including an audit hook (`logging.info` or `audit_trail`). These persist with a `WARN` flag in their metadata. For tools, an additional sub-gate validates input and output schemas as well-formed JSON Schema.

Gate 4: Adversarial validation. The `validation_agent.py` module (121 lines) generates a second, independent test via a separate LLM call. The validating model receives only the artifact source and inlines the function into a fresh test harness with six adversarial assertions targeting empty, `None`, boundary, very-large, negative, unicode, and type-edge inputs. The harness must print `VALIDATION PASSED`.

The four gates are layered because no single one is sufficient. AST catches syntax but not logic. Unit tests catch logic only for cases the original model thought of. ComplianceShield catches dangerous patterns but not behavior. Adversarial validation catches edge cases but is bounded by the validating model’s coverage. Together, they admit only artifacts that survive multiple independent forms of scrutiny.

3.2 Skill-Tool Typing Distinction

The block marker the model emits routes the artifact to a different verification path and invocation pattern. Skills are pure Python functions that operate on data already in memory. They do not touch the network or filesystem. They are imported and called directly by the agent’s generated code. Examples include `validate_iban`, `parse_pain001_fields`, and `compute_moving_average`. Skills are stored in `skills.py/` as one file each. Tools cross a boundary (network, filesystem, external API). They are invoked through `---TOOL CALL---` blocks. The orchestrator parses JSON arguments, dispatches the call, captures the result, and injects it back into the conversation. Examples include `fetch_stock_price`, `read_file`, and `fetch_weather`. Tools are stored in `tools/registry.json`. The two share the first three verification gates. Tools additionally require the schema-validation sub-gate. Both share Gate 4.

The distinction matters for three reasons. Auditability differs. Tool calls are mediated by the orchestrator and logged with arguments and return values. Testability differs. Skills are pure functions with deterministic tests; tools depend on external services with non-deterministic outcomes. Composition differs. Skills are imported and composed naturally inside generated code; tools are dispatched through structured calls that can be mocked or replaced.

3.3 Persistent Stores and Lifecycle Mechanics

Three flat-file stores hold all cross-session state. The skill library (`skills_py/`) holds verified Python skills with metadata headers and a manifest at `skills_py/index.json`. The episodic memory (`memory/episodes.jsonl`) is append-only, embedded for similarity retrieval. The tool registry (`tools/registry.json`) holds schemas and usage counts. A RAG layer (`rag.py`, 287 lines) loads a domain-specific corpus from `rag_corpora/<domain>.json`, embeds it with `sentence-transformers/all-MiniLM-` and injects the two highest-similarity chunks into the system prompt of each task.

Four lifecycle mechanics keep the library accurate and lean. Composition runs before generating new code. The orchestrator retrieves the three highest-similarity skills and asks the model whether the task can be solved by combining them. If so, the composition is verified through the full pipeline and saved with a `composed_from` field. Evolution kicks in when a skill’s `fail_count` reaches two and exceeds its `success_count`. The model diagnoses and patches the function. The patched version is verified, and v1 is archived. Decay is based on a relevance score:

$$r = \frac{\text{success_count}}{1 + \text{days_since_used} \times 0.1}.$$

Below 0.5 over 7 days, a skill is de-prioritised. Below 0.1 over 14 days, it is auto-archived. Failure recovery uses a classifier that grades each task on a 0/1/2 scale. On grade of 1 or below, it categorises the failure into one of five types (`hallucination`, `missing_context`, `logical_inconsistency`, `tool_misuse`, or `stylistic_misalignment`), each with a type-specific recovery prompt.

4 Domain-Agnostic Configuration

The architecture’s behavior on a new domain is determined entirely by configuration. A domain is defined by four files: a target codebase or task source, an exploration task set, a benchmark task set, and a domain configuration YAML specifying ComplianceShield rules, RAG corpus path, audit-relevance vocabulary, and dependency allowlist. To run on a new domain, a user provides these and invokes the orchestrator with the corresponding flags. No agent code changes.

The ComplianceShield rules are loaded at startup. A generic rule set covers universally dangerous patterns such as hard-coded credentials and SSN patterns. Domain-specific rules layer on

top: IBAN and routing numbers for payments, HIPAA identifiers for healthcare, cloud-credential patterns for cloud-infrastructure deployments. The shield logic is unchanged across domains. Only the rules vary.

The bounded dependency allowlist (`config/allowed_dependencies.yaml`) lists Python packages the agent is permitted to install when an authored tool fails Gate 2 with `ModuleNotFoundError`. The default allowlist contains common reputable libraries (`yfinance`, `requests`, `pandas`, `numpy`). The architecture supports unbounded installation. The allowlist is a deliberate policy boundary for production safety.

To be explicit about the boundary between fixed and configurable elements: the orchestration loop, the four-gate verification pipeline, the skill-tool typing distinction, the lifecycle mechanics, the failure-classifier taxonomy, and the embedding-based retrieval are architectural and do not change across domains. The ComplianceShield rules, RAG corpus, exploration and benchmark tasks, target codebase, audit vocabulary, dependency allowlist, and any domain-specific MCP tools are configurable per domain.

5 Models and Technologies

The reported experiment uses Google’s `gemini-2.5-flash-lite` via the `google-genai` client, chosen primarily for free-tier daily token allowance of approximately 1.5 million tokens per day. This made a full cold-warm experiment feasible in a single calendar day. The architecture is model-agnostic. Switching to Claude, GPT-4, or another LLM requires a configuration line change.

Embedding for skill retrieval and episodic similarity uses `sentence-transformers/all-MiniLM-L6-v2`, running locally on CPU. The system has no dependency on a higher-order framework such as LangChain or LlamaIndex. The orchestration layer, verification pipeline, lifecycle mechanics, and MCP client are implemented directly in approximately 4,200 lines of Python distributed across `run.py` (1,629 lines), `mcp_client.py` (306), `rag.py` (287), `compliance.py` (143), `validation_agent.py` (121), and supporting evaluation scripts. The decision not to build atop a larger framework was deliberate. Framework abstractions tend to obscure the verification and lifecycle mechanics this work studies.

6 Demonstrations

We provide three demonstrations: an interactive entry point demonstrating live tool authoring against a domain the agent has not been tuned for, a worked ISO 20022 payment simulation against the framework’s domain-tuned skill library, and a controlled cold-warm experiment whose results appear in Section 7.

6.1 Demonstration A: Interactive Tool Authoring

A single-task interactive entry point (`ask.py`) allows a user to invoke the agent with a natural-language task and watch the architecture work in real time. When a user invokes `python ask.py "what is the current price of AAPL?"`, the agent inspects the existing skill library and tool registry and recognizes that no `fetch_stock_price` capability exists. The agent emits an `---MCP TOOL---` block with name, description, input schema (`ticker: str, days: int = 5`), output schema, function source (using `yfinance`), and a unit test. The orchestrator runs the full four-gate pipeline. All four pass. The tool is registered at `tools/fetch_stock_price.py`. The agent then emits a `---TOOL CALL---` block, the orchestrator dispatches it, and the tool returns `{"current": 270.12, "history": [273.43, 271.06, 267.61, 270.71, 270.12]}`. The agent answers: “AAPL is currently trading at \$270.12, with a 5-day range of \$267.61 to \$273.43.”

This demonstration matters for three reasons. First, the verification pipeline is exercised end-to-end on a tool authored autonomously by the agent. The original cold-warm experiment did not directly observe tool authoring. Second, the agent recognizes when its existing capabilities are insufficient and extends its own toolchain rather than apologizing. Third, the registered tool persists across sessions. A subsequent invocation with a different ticker reuses the same tool.

When verification fails, the orchestrator appends the failure detail to the conversation and re-prompts. The retry budget is three attempts per tool name. For `ModuleNotFoundError` on Gate 2, the orchestrator checks the missing module against the allowlist. If allowed, it runs `pip install` in the active virtualenv and retries Gate 2. This retry does not consume the budget. If the module is not on the allowlist, the failure is permanent and the agent’s retry feedback indicates that a different library should be used.

6.2 Demonstration B: ISO 20022 Cross-Border Payment Simulation

The framework’s payment-domain configuration is exercised through a worked simulation. A euro-denominated cross-border payment from Meridian Ltd in the United Kingdom to Fischer GmbH in Germany for 47,832.50, originating at NatWest and settled through Deutsche Bank. The simulation runs eight stages, each producing real spec-compliant ISO 20022 XML: pain.001 ingestion via `parse_pain001_fields`; IBAN validation via MOD-97 checksum; sanctions screening; FX rate retrieval via `fetch_eur_gbp_rate` from the European Central Bank’s Frankfurter API; pacs.008 construction via `build_pacs008`; pacs.002 acceptance via `build_pacs002_acceptance`; camt.054 notification; and reconciliation. The end-to-end ID E2E-INV-2025-0847 flows unchanged through all four messages, demonstrating the audit-trail property ISO 20022 was designed to support.

The simulation uses ten verified skills accumulated from prior payment-domain runs: three ISO 20022 message constructors, one external-data tool, and six general-purpose utilities. All ten passed the four-gate pipeline. Three carry WARN flags for referencing financial fields without audit-logging calls. Demonstration A and B together bracket the framework’s range. The agent improvising a brand-new capability against a domain it has never seen on one end, and a domain-tuned library executing a complex regulated workflow on the other.

6.3 Demonstration C: Cold-Warm Experimental Design

The hypothesis under test is that an agent with an accumulated skill library outperforms one starting cold. The design is a within-subject comparison on the same benchmark tasks. In the cold condition, the skill library and episodic memory are wiped before the run, and the agent attempts six benchmark tasks with no prior procedural knowledge. In the warm condition, the wipe is followed by twenty exploration tasks intended to populate the library. The same six benchmark tasks then run with whatever skills survived verification.

The benchmark targets the open-source FastAPI codebase. Tasks are organised into three tiers. Tier 1 (structural retrieval) reads the codebase and reports a fact, such as which modules import which Starlette names or what methods `APIRoute` defines. Tier 2 (pattern replication) writes new code that follows an existing convention precisely. Tier 3 (novel synthesis) identifies a pattern repeated across files and abstracts a reusable skill or refactor. Six tasks total, two per tier.

The exploration set comprises twenty single-function ISO 20022 and ACH utility tasks: cents-to-dollars conversion, IBAN normalisation, ABA-checksum calculation, BIC-format validation. The exploration tasks were drawn from the payment domain rather than the FastAPI domain. This is a domain mismatch with the benchmark. The warm condition’s accumulated skills are largely irrelevant to direct reuse, isolating the question of whether the architecture provides benefit through channels other than skill substitution.

Each task is scored 0 to 3 by an LLM-as-judge using the same Gemini model with a different prompt. One-point bonuses are available for executable code, real-file references, and verified-skill invocation, capped at 3.

7 Findings and Discussion

7.1 Quantitative Results

Tier	Cold avg	Warm avg	Difference
Tier 1 (Structural Retrieval)	1.50	2.50	+1.00
Tier 2 (Pattern Replication)	2.00	2.50	+0.50
Tier 3 (Novel Synthesis)	2.50	3.00	+0.50
Overall	2.00	2.67	+0.67

Table 1: Cold-warm benchmark scores by tier (n=6 benchmark tasks). The relative improvement is 33.3 percent.

Six skills were verified during exploration. One (`abstract_collections_abc_imports`) was directly invoked during the warm benchmark and scored 3.0 out of 3.0. No skill compositions were observed. In the cold condition, the failure-classifier distribution was `missing_context` (3), `hallucination` (2), `tool_misuse` (1). Recovery succeeded on 50 percent of attempts. In the warm condition, the distribution was `missing_context` (2), `hallucination` (2), `tool_misuse` (1). Recovery succeeded on 20 percent of attempts. The reduction in `missing_context` from cold to warm is consistent with the system prompt being more informative in the warm condition. Additional skills and episodes are present in context even when those skills are not directly invoked.

Sample size is small (n=6 benchmark tasks) and results derive from a single experimental run. We interpret these numbers as directional rather than statistically significant.

7.2 The Multi-Channel Benefit Hypothesis

The central empirical finding is that warm performance exceeded cold performance across all three task tiers, including the structural-retrieval tier (Tier 1, +1.00), where accumulated skills should have been least directly applicable. A naive reading of the cold-warm hypothesis would predict the largest gains where direct skill reuse is possible (Tier 2 and Tier 3) and the smallest gains on Tier 1. The observed pattern is the inverse. Combined with the observation that only one of six accumulated skills was directly invoked during the warm benchmark, this points to the architecture’s benefit being multi-channel rather than concentrated in skill substitution.

We identify three plausible channels. Channel 1 is episodic memory. Every new task has the three most semantically similar past episodes injected into its system prompt. In the warm condition, those episodes include twenty exploration episodes, giving the model a richer working context. Channel 2 is tool-call discipline. The non-empty skill manifest in the warm condition appears to nudge the model toward the action-as-Python-function pattern. Logs show warm tasks were more likely to invoke `read_file` before generating output. Channel 3 is direct skill reuse. This is the channel the architecture was explicitly designed to produce, and the one that did the least work in this run. With a benchmark and exploration set drawn from the same domain, we would expect this channel to contribute substantially more.

That the warm condition outperforms cold by 33 percent even when the accumulated procedural knowledge is from the wrong domain is, in a sense, a stronger result than a perfectly matched domain would have produced. It suggests the architecture has value even when the skill library is not perfectly aligned with the next task. An ablation study isolating each channel is identified as immediate future work.

7.3 Verification-Pipeline Behaviour

Across the run, six artifacts were admitted and four were rejected at Gate 4, a 60 percent admission rate. Gate 1 failures occurred most often when output was being truncated mid-generation due to `max_output_tokens` being set too low (initially 2,000). After raising the limit to 8,000, AST failures reduced substantially. Gate 2 failures occurred when the model wrote tests using `assert` without the explicit `print("TEST PASSED")` marker. We treat false negatives at this gate as preferable to

false positives. Gate 3 issued one BLOCK on a payment-domain skill containing a literal nine-digit number, and WARN flags on three skills referencing financial fields without audit hooks. Gate 4 caught four skills that had passed Gates 1 through 3. The validating model identified edge cases the original test did not cover. The validating model was the same as the authoring model, which is a conflict-of-interest concern. A stronger validating model would likely catch more failures. The 60 percent admission rate is evidence that the pipeline is calibrated appropriately. Strict enough to reject obvious failures, loose enough not to suppress skill formation.

7.4 Qualitative Evidence from Live Tool Authoring

Beyond the controlled cold-warm experiment, the interactive `ask.py` entry point produced qualitative evidence on a different question: whether tool authoring works end-to-end in a domain the agent has not been tuned for. Across rehearsal runs, the agent reliably authored a working `fetch_stock_price` tool on the first or second attempt. First-attempt failures most commonly resolved on the second attempt after the bounded dependency-resolution path installed `yfinance` from the allowlist. A separate set of rehearsals against a weather-domain task showed the agent authoring a working `fetch_weather` tool using `urllib` and the Open-Meteo API. In both cases the four verification gates ran and produced visible pass/fail signals before the tool was registered. The honest characterization is that tool authoring works reliably enough to demonstrate live. We decline to report a precise success rate from these rehearsals given the small sample.

8 Responsible AI Considerations

8.1 Hallucination Mitigation

The verification pipeline rejects functions that fail their own unit tests or adversarial validation. The episodic memory layer injects records of similar past tasks into the system prompt. The RAG layer surfaces authoritative reference documents directly, reducing reliance on potentially-outdated training data.

8.2 Privacy and PII

The ComplianceShield rejects skills containing hard-coded PII patterns. The regex-based approach lacks AST-level taint analysis. Production deployment would require integration with a real PII detector and a quarantine workflow for borderline artifacts.

8.3 Bias

The LLM-as-judge inherits the underlying model’s biases. The five-category failure taxonomy may misclassify edge cases. The all-MiniLM-L6-v2 embedding is English-centric, potentially impacting retrieval relevance for non-English identifiers. The benchmark and exploration tasks are technical and do not involve decisions about individuals, so no individual-fairness concerns arise directly.

8.4 Safety and Security

The verification subprocess has a 30-second timeout. The ComplianceShield blocks hard-coded routing numbers. The bounded dependency allowlist restricts what the agent can install. Production deployment would require containerization (such as E2B or Modal sandboxes), AST-level taint tracking, and a vetted package list. The system as implemented is intended for offline development against synthetic codebases, not live integration with payment infrastructure.

9 Limitations and Future Work

We identify six limitations and the corresponding immediate future work.

9.1 Sample Size and Single-Model Evaluation

The reported cold-warm result is from six benchmark tasks in a single experimental run. Replication on the full fifteen-task benchmark with multiple runs per condition is necessary to convert directional evidence into a quantitative claim. All runs used `gemini-2.5-flash-lite`. The verification pipeline, failure classifier, and judge are correlated. A multi-model evaluation would tighten the result.

9.2 Domain Mismatch and Second-Domain Replication

Payment exploration paired with a FastAPI benchmark unintentionally isolated context-quality benefit. A domain-matched run is the most immediate next experiment. The architecture is domain-agnostic by design, and we have qualitative transferability evidence from `ask.py`, but we have not run a controlled cold-warm experiment in a second domain. This is the most direct evidence to close RQ4.

9.3 Ablation Study

The three benefit channels are not separately tested. An ablation series with each context layer disabled (episodic memory, skill index, RAG, tool manifest) would attribute the 33.3 percent gain to specific architectural components.

9.4 ComplianceShield Hardening

The static, regex-based ComplianceShield is appropriate for research and inappropriate for production without AST-level taint tracking and integration with a real PII detector. Two further directions are worth flagging. Skill federation across deployments would let an institution joining a network inherit verified, anonymised skills from prior institutions, a substantial practical benefit with non-trivial questions about trust and provenance. Stable-across-evolution skill identifiers would let composed skills bind to a stable identifier while transparently routing to the latest verified version, allowing the library to evolve without breaking dependents.

10 Conclusion

We presented a domain-agnostic framework in which a code-acting agent autonomously authors, verifies, and accumulates a permanent library of executable Python skills and external tools as a by-product of solving tasks. The architecture extends open-ended action accumulation with a four-gate verification pipeline, a typed skill-tool distinction with separate verification semantics, lifecycle mechanics for composition, evolution, and decay, an episodic memory layer, and an external configuration interface that allows the framework to be pointed at any domain by supplying configuration rather than modifying agent code.

We provided three forms of evidence. The cold-warm experiment on FastAPI produced directional quantitative evidence of a 33.3 percent improvement, with a multi-channel pattern suggesting the architecture’s benefit derives from accumulated context quality and tool-call discipline as well as direct skill reuse. The interactive `ask.py` entry point demonstrated live tool authoring. The agent recognized a missing capability, authored an MCP tool to fill it, passed it through the four-gate pipeline, and used it on the same task. The ISO 20022 simulation demonstrated the framework executing a realistic eight-step financial workflow against its tuned skill library.

The broader thesis is that procedural knowledge in code-acting agents should be persistent, verified, and lifecycle-managed rather than ephemeral and conversational. The cold-warm result is one piece of evidence in support of that thesis. The more substantial test will come from longer-running deployments against real codebases, where lifecycle mechanics have time to operate at scale and the configuration interface is exercised by users beyond the original authors.

References

- [1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.
- [2] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366.
- [3] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., & Anandkumar, A. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.
- [4] Nguyen, D., Lai, V., Yoon, S., Rossi, R., Zhao, H., Zhang, R., Mathur, P., Lipka, N., Wang, Y., Bui, T., Derroncourt, F., & Tran, T. (2024). DynaSaur: Large Language Agents Beyond Predefined Actions. arXiv:2411.01747.
- [5] Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., & Ji, H. (2024). Executable Code Actions Elicit Better LLM Agents. arXiv:2402.01030.

- [6] Hugging Face. (2024). smolagents: A Smol Library to Build Great Agentic Systems. GitHub: [huggingface/smolagents](https://github.com/huggingface/smolagents).
- [7] Park, J. S., O’Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. In *UIST ’23*.
- [8] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*.
- [9] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *NeurIPS 2023*.
- [10] SWIFT. (2025). Cross-border Payments and Reporting Plus (CBPR+): End of Coexistence Period. SWIFT Standards Release Documentation.
- [11] Industry survey reporting that 83 percent of payment modernisation projects experience delays, 87 percent exceed budgets, and 83 percent fail to deliver original specifications. Cited in payment-industry trade press, 2025.
- [12] NACHA. (2024). Operating Rules and Guidelines, 2024 Edition. National Automated Clearing House Association.
- [13] International Organization for Standardization. ISO 20022 Message Definition Registry. <https://www.iso20022.org/iso-20022-message-definitions>